

Q1: Simple Linear Regression (OLS without library)

```
# Manual calculation of slope and intercept
x = [1, 2, 3, 4, 5]
y = [2, 4, 5, 4, 5]

n = len(x)
sum_x = sum(x)
sum_y = sum(y)
sum_xy = sum([x[i]*y[i] for i in range(n)])
sum_x2 = sum([x[i]**2 for i in range(n)])

m = (n*sum_xy - sum_x*sum_y) / (n*sum_x2 - sum_x**2)
c = (sum_y - m*sum_x) / n

print(f"Slope (m) = {m}")
print(f"Intercept (c) = {c}")
y_pred = [m*xi + c for xi in x]
print("Predicted y values:", y_pred)
```

Q2: Simple Linear Regression (with library)

```
from sklearn.linear_model import LinearRegression
import numpy as np

x = np.array([1, 2, 3, 4, 5]).reshape(-1,1)
y = np.array([2, 4, 5, 4, 5])

model = LinearRegression()
model.fit(x, y)

print(f"Slope (m) = {model.coef_[0]}")
print(f"Intercept (c) = {model.intercept_}")
y_pred = model.predict(x)
print("Predicted y values:", y_pred)
```

Q3: Multiple Linear Regression (with library)

```
X = np.array([[1,2],[2,3],[3,4],[4,5],[5,6]])
y = np.array([2,3,4,5,6])
```

```

model = LinearRegression()
model.fit(X, y)

print(f"Coefficients = {model.coef_}")
print(f"Intercept = {model.intercept_}")
y_pred = model.predict(X)
print("Predicted y values:", y_pred)

```

Q4: Multiple Linear Regression (without library)

```

X = np.array([[1,2],[2,3],[3,4],[4,5],[5,6]])
y = np.array([2,3,4,5,6])

X_b = np.c_[np.ones((X.shape[0],1)), X]
B = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
print(f"Intercept = {B[0]}")
print(f"Slopes = {B[1:]}")
y_pred = X_b.dot(B)
print("Predicted y values:", y_pred)

```

Q5: Gradient Descent (without library)

```

x = np.array([1,2,3,4,5])
y = np.array([2,4,5,4,5])

m, c = 0.0, 0.0
alpha = 0.01
n = len(x)
epochs = 1000

for _ in range(epochs):
    y_pred = m*x + c
    dm = (-2/n)*sum(x*(y - y_pred))
    dc = (-2/n)*sum(y - y_pred)
    m -= alpha*dm
    c -= alpha*dc

print(f"Slope (m) = {m}")
print(f"Intercept (c) = {c}")
y_pred = m*x + c
print("Predicted y values:", y_pred)

```

Q6: Gradient Descent (with library)

```
from sklearn.linear_model import SGDRegressor
x = np.array([1,2,3,4,5]).reshape(-1,1)
y = np.array([2,4,5,4,5])

model = SGDRegressor(max_iter=1000, learning_rate='constant', eta0=0.01,
random_state=42)
model.fit(x, y)

print(f"Slope (m) = {model.coef_[0]}")
print(f"Intercept (c) = {model.intercept_[0]}")
y_pred = model.predict(x)
print("Predicted y values:", y_pred)
```

Q7: Polynomial Regression (with library)

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,3,4,5]).reshape(-1,1)
y = np.array([1,4,9,16,25])

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(x)
model = LinearRegression()
model.fit(X_poly, y)

print(f"Coefficients = {model.coef_}")
print(f"Intercept = {model.intercept_}")
y_pred = model.predict(X_poly)

plt.scatter(x, y, color='red')
plt.plot(x, y_pred, color='blue')
plt.show()
```

Q8: Polynomial Regression (without library)

```
x = np.array([1,2,3,4,5])
y = np.array([1,4,9,16,25])
degree = 2
X = np.vander(x, N=degree+1, increasing=True)
```

```

B = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
print(f"Coefficients = {B}")
y_pred = X.dot(B)

import matplotlib.pyplot as plt
plt.scatter(x, y, color='red')
plt.plot(x, y_pred, color='blue')
plt.show()

```

Q9: KNN Algorithm (with library)

```

from sklearn.neighbors import KNeighborsClassifier
import numpy as np
X = np.array([[1,2],[2,3],[3,3],[6,5],[7,7],[8,6]])
y = np.array([0,0,0,1,1,1])
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)
new_data = np.array([[4,4],[5,5]])
predictions = knn.predict(new_data)
print(predictions)

```

Q10: KNN Algorithm (without library)

```

import numpy as np
from collections import Counter
X = np.array([[1,2],[2,3],[3,3],[6,5],[7,7],[8,6]])
y = np.array([0,0,0,1,1,1])
new_data = np.array([[4,4],[5,5]])

def euclidean_distance(a,b):
    return np.sqrt(np.sum((a-b)**2))

def knn_predict(X_train, y_train, x_test, k=3):
    predictions = []
    for point in x_test:
        distances = [euclidean_distance(point, x_train) for x_train in X_train]
        k_indices = np.argsort(distances)[:k]
        k_labels = [y_train[i] for i in k_indices]
        most_common = Counter(k_labels).most_common(1)[0][0]
        predictions.append(most_common)
    return predictions

predictions = knn_predict(X, y, new_data, k=3)
print(predictions)

```

Q11: Agglomerative Clustering

```
from sklearn.cluster import AgglomerativeClustering
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[1,2],[2,3],[3,3],[6,5],[7,7],[8,6]])
agg_clust = AgglomerativeClustering(n_clusters=2, affinity='euclidean',
linkage='ward')
labels = agg_clust.fit_predict(X)
print(labels)
plt.scatter(X[:,0], X[:,1], c=labels, cmap='rainbow')
plt.show()
```

Q12: K-Means Clustering

```
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[1,2],[2,3],[3,3],[6,5],[7,7],[8,6]])
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
print(labels)
print(centroids)
plt.scatter(X[:,0], X[:,1], c=labels, cmap='rainbow')
plt.scatter(centroids[:,0], centroids[:,1], color='black', marker='x', s=100)
plt.show()
```

Q13: Apriori Algorithm

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

dataset = [
    ['milk', 'bread', 'butter'],
    ['bread', 'butter', 'jam'],
    ['milk', 'bread'],
    ['milk', 'bread', 'butter', 'jam'],
    ['bread', 'butter']
]
all_items = sorted(set(item for transaction in dataset for item in transaction))
encoded_vals = [[1 if item in transaction else 0 for item in all_items] for
transaction in dataset]
```

```
df = pd.DataFrame(encoded_vals, columns=all_items)

frequent_itemsets = apriori(df, min_support=0.6, use_colnames=True)
print(frequent_itemsets)
rules = association_rules(frequent_itemsets, metric='confidence',
min_threshold=0.7)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

This document contains all 13 ML programs (Q1-Q13) with explanations and optional plots for visualization.